partnr

Build a Serverless Event-Driven Image Processing Pipeline with AWS Lambda and SQS

Back

Mandatory

Domain

Backend Development

Cloud Computing

Skills

API Development | Automation | Cloud Computing | Configuration Management | Deep Learning | Infrastructure as Code Security | Public Relations | Serverless | Solution Architecture | aws-dynamodb | aws-lambda | aws-s3 | aws-sqs

Difficulty

Hard

Tools

Amazon SQS | AWS Cloud Services | Cloudformation | Docker | DynamoDB | Lambda | LocalStack | Node.Js | Python | S3 | Serverless | Terraform

Industries

E-commerce | Technology

Pending Submission

Please submit your work when ready.

TIME REMAINING

20h

Deadline: **30 May 2026, 04:59 pm**

Overview | Instructions | Resources | Submit

Objective

Build a resilient and scalable serverless image processing pipeline using AWS. This task will challenge you to design an event-driven system that automatically processes images upon upload. You will gain hands-on experience with AWS Lambda for serverless compute, Amazon S3 for object storage and event notifications, and Amazon SQS for asy communication and message queuing. You will learn to configure event triggers, write

Report Issue

stateless Lambda functions, manage application state through object storage, and

partnr

available, cost-effective cloud solutions, which are critical skills for high-paying remote cloud engineering roles.

Description

Background

Image processing is a common requirement in many modern applications, from e-commerce to social media. Building robust, scalable, and cost-effective image processing pipelines is crucial. Traditional approaches often involve managing servers, which can be expensive and complex to scale. This task challenges you to implement a serverless solution on AWS, leveraging event-driven architecture to automatically handle image transformations. This pattern is highly sought after for its efficiency, scalability, and reduced operational overhead.

Implementation Guidelines

Serverless Design Principles

Embrace the serverless paradigm by designing stateless Lambda functions. Functions should be single-purpose and execute quickly. Leverage AWS managed services for storage, messaging, and databases to minimize operational burden.

Event-Driven Architecture

The system should be truly event-driven. An event (image upload to S3) should trigger a downstream process (Lambda function) which then produces another event (SQS message) for subsequent processing.

Idempotency and Retries

Consider how your Lambda functions will handle retries and ensure operations are idempotent where possible to prevent unintended side effects from duplicate invocations. SQS's visibility timeout and retry mechanisms should be understood.

IAM Roles and Least Privilege

Design granular IAM roles for each Lambda function with only the necessary permissions (least privilege). Avoid granting overly broad permissions.

Error Handling and Dead-Letter Queues (DLQs)

Implement robust error handling for critical steps in the processing pipeline. Utilize Dead-

partnr

Infrastructure as Code (IaC)

All AWS resources must be defined and deployed using Infrastructure as Code (IaC) tools. This ensures reproducibility, version control, and simplifies deployment and tearing down of environments.

Local Development and Testing

Provide a clear strategy for local development and testing, ideally using tools like Localstack to simulate AWS services, enabling rapid iteration without incurring AWS costs.

Implementation Details

Project Structure

Organize your project with clear separation for IaC templates, Lambda function code, and test files.

```
.
├── infra/                      # Infrastructure as Code files (CloudFormation, Serverless, Ter
│   ├── main.yaml               # or main.tf, serverless.yml, etc.
│   └── outputs.json            # (if applicable for CloudFormation/CDK)
├── src/
│   ├── image_processor/        # ImageProcessorLambda code
│   │   ├── app.py              # or index.js
│   │   ├── requirements.txt    # or package.json
│   │   └── tests/
│   │       └── test_processor.py
│   ├── metadata_updater/       # MetadataUpdaterLambda code
│   │   ├── app.py              # or index.js
│   │   ├── requirements.txt    # or package.json
│   │   └── tests/
│   │       └── test_updater.py
├── tests/                      # End-to-end integration tests
│   └── e2e_test.py             # Python script using boto3 for example
├── .env.example                # Example environment variables
├── Dockerfile                  # If custom runtime or complex dependencies
├── docker-compose.yml          # For Localstack setup
└── README.md
```

AWS Resource Naming Conventions

Use descriptive, unique names for your resources, ideally including a project name and stage (e.g., `image-pipeline-input-bucket-dev-yourname` , `image-pipeline-processor-lambda-dev-yourname`).

S3 Buckets

partnr

AWS Lambda Functions

ImageProcessorLambda

1. **Runtime**: Python 3.x or Node.js 18.x (or newer).
2. **Handler**: `app.handler` (or `index.handler`).
3. **Environment Variables**:
    1. `TARGET_WIDTH` : e.g., "200" (string representation of an integer).
    2. `WATERMARK_TEXT` : e.g., "© MyCompany".
    3. `SQS_QUEUE_URL` : URL of the `ImageProcessedQueue` .
    4. `DLQ_QUEUE_URL` : URL of the `DLQProcessorErrors` .
    5. `PROCESSED_BUCKET_NAME` : Name of the `processed-image-bucket` .
4. **Sample Python Lambda Handler Signature**:

```
import json
import os
import boto3
from PIL import Image, ImageDraw, ImageFont # Example for image processing

s3_client = boto3.client('s3')
sqs_client = boto3.client('sqs')

def handler(event, context):
    # #### Step 1: Parse S3 event
    # Extract bucket name and object key from event
    # #### Step 2: Download image
    # s3_client.download_file(...)
    # #### Step 3: Validate and process image
    # Use PIL or similar library for resizing and watermarking
    # #### Step 4: Upload processed image
    # s3_client.upload_file(...)
    # #### Step 5: Publish message to SQS or DLQ
    # sqs_client.send_message(...)
    return {
        'statusCode': 200,
        'body': json.dumps('Image processing initiated successfully!')
    }
```

5. **Dependencies**: Include necessary image processing libraries (e.g., `Pillow` for Python, `sharp` for Node.js - ensure packaging for Lambda).

MetadataUpdaterLambda

1. **Runtime**: Python 3.x or Node.js 18.x (or newer).
2. **Handler**: `app.handler` (or `index.handler`).

3. **Environment Variables**:

partnr

```
import json
import os
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ.get('DYNAMODB_TABLE_NAME'))

def handler(event, context):
    for record in event['Records']:
        # #### Step 1: Parse SQS message
        # record['body'] contains the SQS message
        # #### Step 2: Extract metadata
        # Parse JSON from SQS message body
        # #### Step 3: Store in DynamoDB
        # table.put_item(Item={...})
    return {
        'statusCode': 200,
        'body': json.dumps('Metadata updated successfully!')
    }
```

Amazon SQS Queues

Define `ImageProcessedQueue`, `DLQProcessorErrors`, and `DLQProcessedMessages`. Ensure `ImageProcessedQueue` has `DLQProcessedMessages` as its Dead-Letter Queue with a `maxReceiveCount` (e.g., 5).

Amazon DynamoDB Table

Define `ImageMetadataTable` with `originalKey` (String type) as the Partition Key.

```
{
  "TableName": "ImageMetadataTable",
  "KeySchema": [
    { "AttributeName": "originalKey", "KeyType": "HASH" }
  ],
  "AttributeDefinitions": [
    { "AttributeName": "originalKey", "AttributeType": "S" }
  ],
  "ProvisionedThroughput": {
    "ReadCapacityUnits": 5,
    "WriteCapacityUnits": 5
  }
}
```

Local Development with Docker Compose and Localstack

Provide a `docker-compose.yml` to run Localstack and expose necessary AWS service

partnr

```yaml
version: '3.8'
services:
  localstack:
    container_name: "${LOCALSTACK_HOST:-localstack_main}"
    image: localstack/localstack
    ports:
      - "127.0.0.1:4510-4559:4510-4559"  # external service port range
      - "127.0.0.1:4566:4566"            # LocalStack Gateway
    environment:
      - DEBUG=${DEBUG:-0}
      - DOCKER_HOST=unix:///var/run/docker.sock
      - SERVICES=s3,lambda,sqs,dynamodb
      - AWS_DEFAULT_REGION=us-east-1
      - AWS_ACCESS_KEY_ID=test
      - AWS_SECRET_ACCESS_KEY=test
    volumes:
      - "${LOCALSTACK_VOLUME_DIR:-./localstack/volume}:/var/lib/localstack"
      - "/var/run/docker.sock:/var/run/docker.sock"
```

Instructions for using AWS CLI with Localstack should be provided (e.g., `aws --endpoint-url=http://localhost:4566 s3 mb s3://my-local-bucket`).

Deployment

Use an IaC tool (CloudFormation, Serverless Framework, CDK, Terraform). Instructions for deploying all resources and Lambda functions.

Testing

Include unit tests for Lambda logic (image resizing, message parsing). Provide an end-to-end integration test script (e.g., a Python script using `boto3`) to:

1. Upload a test image to the `input-image-bucket` (localstack or actual).
2. Poll `processed-image-bucket` for the resized image.
3. Poll `ImageMetadataTable` in DynamoDB for the corresponding entry.
4. Verify content/structure of SQS messages (if possible to consume and re-queue without disrupting flow).

Submission Instructions

Your submission must include:

1. **Application Code**: All source code for your Lambda functions and any helper utilities.
2. **Infrastructure as Code**: All files required to deploy your AWS resources (e.g., CloudFormation templates, Serverless.yml, Terraform files, CDK code).

3. **`README.md`** : A comprehensive README file documenting your project (see below for

partnr

5. **`.env.example`** : Documenting all required environment variables.
6. **`tests/`** : Directory containing unit and end-to-end integration tests.

For all submissions, `docker-compose.yml` is **REQUIRED** for local development/simulation of AWS services. Ensure your solution can be set up and deployed locally using `docker-compose up` with Localstack, and then deployed to a real AWS account.

Optional (bonus) artifacts:

1. `ARCHITECTURE.md` : Detailed architecture diagrams and design decisions.
2. Screenshots of processed images or DynamoDB table entries.
3. A short video demo of the pipeline in action (uploading an image, seeing it processed, verifying DynamoDB entry).

The `README.md` should include:

1. Project Title and Overview.
2. Setup and Local Development Instructions (including how to use `docker-compose` with Localstack).
3. Deployment Instructions to AWS.
4. Usage Guide (how to trigger the pipeline, how to check results).
5. API Endpoints (N/A for this event-driven task, but general structure).
6. Testing Instructions and Results.
7. Assumptions and Trade-offs.
8. Future Improvements.

Evaluation Overview

Your submission will be rigorously evaluated for functionality, architectural design, code quality, and documentation. We will run automated tests to verify that all core requirements are met, including the correct triggering of Lambda functions, accurate image processing, proper SQS messaging, and reliable data persistence in DynamoDB. Your code will undergo automated code analysis to assess its structure, adherence to best practices, error handling, and security considerations. Additionally, expert reviewers will manually assess your project's overall design, the clarity and completeness of your documentation, and the robustness of your solution, particularly focusing on your IaC implementation and local development setup. A strong emphasis will be placed on demonstrating a fully functional, event-driven pipeline that is easily deployable and testable.

Common Mistakes To Avoid

1. **Insufficient IAM Permissions**: Lambda roles that lack permissions for S3, SQS, or

using IAM roles or environment variables.
3. **Lack of Error Handling**: Functions failing silently or without proper logging for failed image processing.
4. **Incorrect Event Configuration**: S3 event notifications not correctly triggering Lambda, or Lambda not correctly consuming SQS messages.
5. **Monolithic Lambda Functions**: Trying to do too much in a single Lambda, violating serverless best practices.
6. **Missing or Incomplete IaC**: Not defining all resources with IaC, or providing IaC that doesn't deploy correctly.
7. **Unclear Local Development Setup**: Difficulty in reproducing the environment locally using `docker-compose` and Localstack.
8. **Race Conditions/Idempotency Issues**: Not considering how concurrent uploads or retries might affect data integrity.
9. **Large Lambda Deployment Packages**: Including unnecessary libraries, leading to slow cold starts.

FAQ

1. **Q: Which programming language should I use for Lambda functions?** A: You are free to choose either Python (e.g., `boto3` + `Pillow`) or Node.js (e.g., `aws-sdk` + `sharp`). Use the one you are most comfortable with.
2. **Q: Can I use AWS CDK/Serverless Framework/Terraform for IaC?** A: Yes, any of these industry-standard IaC tools are acceptable. Ensure your IaC is fully functional and deployable.
3. **Q: Do I need to implement a frontend?** A: No, this task is purely backend and infrastructure-focused. You can use the AWS CLI or `boto3` (Python) / `aws-sdk` (Node.js) to simulate image uploads.
4. **Q: How do I test the S3 event trigger locally?** A: With Localstack running via `docker-compose`, you can use the `aws --endpoint-url=http://localhost:4566 s3 cp <local-image> s3://input-image-bucket` command to simulate an upload, which should trigger your local Lambda.
5. **Q: What image processing operations are required?** A: The core requirements are resizing to a specified width and applying a text watermark. You can add more for bonus points, but focus on these two.
6. **Q: How should I handle secrets like API keys for potential third-party services (if any)?** A: For this task, there are no external API keys required. For AWS access, ensure your Lambda functions use IAM roles with least privilege. For local development with Localstack, standard `AWS_ACCESS_KEY_ID=test` and `AWS_SECRET_ACCESS_KEY=test` are used.
7. **Q: What should the `unique_id` be in bucket names?** A: You can use a random string, a timestamp, or your GitHub username to ensure global uniqueness for S3 bucket names.

8. **Q: Is comprehensive error logging important?** A: Yes, structured logging (e.g., JSON

partnr

Core Requirements

1. An S3 bucket named `input-image-bucket-<unique_id>` must exist for raw image uploads.
2. An S3 bucket named `processed-image-bucket-<unique_id>` must exist for processed images.
3. Both S3 buckets must be configured with appropriate access policies for Lambda functions.
4. An AWS Lambda function named `ImageProcessorLambda` must be deployed and configured to run with either Python 3.x or Node.js 18.x (or newer).
5. `ImageProcessorLambda` must be automatically triggered by `s3:ObjectCreated:*` events in the `input-image-bucket`.
6. The Lambda function's IAM role must have permissions to read from `input-image-bucket`, write to `processed-image-bucket`, and publish messages to `ImageProcessedQueue` and `DLQProcessorErrors`.
7. `ImageProcessorLambda` must include environment variables for configuration (e.g., `TARGET_WIDTH`, `WATERMARK_TEXT`, `SQS_QUEUE_URL`, `DLQ_QUEUE_URL`, `PROCESSED_BUCKET_NAME`).
8. `ImageProcessorLambda` must download the newly uploaded image from the `input-image-bucket`.
9. The function must validate the uploaded file type (accepting JPG/JPEG and PNG only).
10. For valid images, the function must resize the image to the `TARGET_WIDTH` (e.g., 200 pixels) while maintaining its aspect ratio.
11. A text watermark (specified by `WATERMARK_TEXT`) must be applied to the resized image.
12. The processed image must be uploaded to the `processed-image-bucket` with a filename format like `resized_<original_key>`.
13. An SQS Standard Queue named `ImageProcessedQueue` must be created.
14. Upon successful image processing and S3 upload, `ImageProcessorLambda` must publish a JSON message to `ImageProcessedQueue` containing `originalKey`, `processedKey`, `timestamp`, `status` (`SUCCESS`), and `processingDetails` (e.g., original size, new size, duration).
15. An SQS Standard Queue named `DLQProcessorErrors` must be created.
16. If `ImageProcessorLambda` encounters an error during image download, processing, or upload, it must publish a JSON error message to `DLQProcessorErrors` including `originalKey`, `errorType`, `errorMessage`, and `timestamp`.
17. An AWS Lambda function named `MetadataUpdaterLambda` must be deployed (Python 3.x or Node.js 18.x).
18. `MetadataUpdaterLambda` must be configured to consume messages from `ImageProcessedQueue`.

19. The Lambda function's IAM role must have permissions to read from

partnr

`originalKey` as its Partition Key.

21. `MetadataUpdaterLambda` must parse messages from `ImageProcessedQueue` and persist the `originalKey`, `processedKey`, `timestamp`, `status`, and `processingDetails` into `ImageMetadataTable`.
22. `ImageProcessedQueue` must be configured with `DLQProcessedMessages` as its Dead-Letter Queue with an appropriate `maxReceiveCount`.
23. An SQS Standard Queue named `DLQProcessedMessages` must be created.
24. All AWS resources (S3 buckets, Lambda functions, SQS queues, DynamoDB table, IAM roles) must be defined using a supported IaC tool (AWS CloudFormation, Serverless Framework, AWS CDK, or Terraform).
25. The IaC template(s) must be deployable via a single command or script.
26. A `docker-compose.yml` file must be provided for local development/testing using Localstack to simulate AWS services.
27. Localstack must simulate S3, SQS, Lambda, and DynamoDB.
28. Clear instructions for setting up and running the local environment and deploying the serverless application to Localstack must be included in `README.md`.
29. Unit tests for the core logic of `ImageProcessorLambda` and `MetadataUpdaterLambda` (e.g., image resizing, message parsing) must be provided.
30. A script or commands for end-to-end integration testing against the deployed (Localstack or actual AWS) pipeline must be provided to verify image upload, processing, SQS messaging, and DynamoDB updates.

---

partnr